Loup M. Marquez Luttringer – Project Manager

Matheus Lopez – Back-End & Front-End Developer

Noah Gibelli – AI Developer

Version: v1.0

Date: October 10, 2025

Horus Project – LLM Implementation Justification

(Valorizing Technology)

Point 1 – A – Valorize Technology

Choosing a Language to run our LLM part of the code has been a truly serious subject. Currently, we have found that if we wanted to launch an LLM, we have two very popular languages that can satisfy us: C++ and Python. These languages are probably the most powerful on the market. But the problem is that they are very different. It represents two different coding, two different computer system usage. But in the end, we have to keep only one language. So, we have done an analysis to determine the poor and cons for both of these languages. To begin, the syntax complexity was so different for both of them. We can see that we can write 15 lines with Python to reproduce what can a C++ program that contains 150 lines. C++ is good when you want to have control over aspects like memory management, speed and also being close to the metal. C++ is a very robust language and cool when you pay attention to technical aspects. But when you have to implement it as fast and easy as possible, Python is the best opportunity. Python gives you opportunity to think about your features and how you could innovate. I went from “how do I manage this buffer?” to “let’s run a model with this prompt and see if it works” in a single afternoon.

Also, in terms of AI, Python language is very furnished with TensorFlow, PyTorch or also scikit-learn. And it’s always interesting to see how fast Python compiler can react. With all of these libraries, Python can react in time it takes for a C++ compiler to return an error about missing a semicolon. A large library supports easy to handle. Python Is a language with who you can do a prototype with an LLM very fast in an hour, while you have to setting up the build environment is C++. And with a reduced number of code lines, it’s easy to understand the code and maintain it.

But the problem Is that when you want to inject a large number of code lines inside Python loops, Python kernel can be slower that a C++ kernel and requires a lot of memory depending on the dataset you used with the LLM when you want to train it for example. Also, we have to remember that a lot of practices in Python like matrix multiplications or also GPU kernels, use a C or C++ backend or are just optimized libraries that come from C++ originally. You use C++ with python even if you don’t know. When you have to switch to C++, it’s because you need power at a particular moment like: Real-time processing (drones) or places where memory is tight. C++ is good if you build an algorithm where each second of execution counts. But if you want to build an algorithm as fast as possible or your code will be pushed in production with performance instability, Python can be a better option. But after all, nothing stops you from combining them for example if a part of your program uses C++ to handle memory-hungry user cases. And C++has a static typing so you will encounter less errors at the execution.

Seeing these several poor and cons, we choose to use Python language to have a code compatible with every infrastructure and easy to implement. Today, with the usage of GPU cards, Python can easily redirect code on them and has a large number of libraries to do it. If the code can be redirected on a GPU, having a powerful C++ kernel is less important. But, if sometimes we need to be closer to C++ performance we can still use libraries like Cython that is a compiled C++ library in Python.